Agent Hub, how I give my AI tools shared memory

Updated October 30, 2026, two months after the original post. I've added a skill for saving work, shortened the files agents read on every task, and clarified where finished documents belong. The article and setup instructions below reflect the current version. See the template changelog for the detailed changes.

I use Codex, Claude, and Cursor for research, writing, and project work. Switching between them meant explaining the same projects and decisions again. Something I worked through with one tool rarely carried over to the next.

I've been using a local folder of Markdown files to keep that context. I call it Agent Hub. Each agent reads the notes relevant to its task and saves what changed when it's done. I keep the folder in a GitHub repo so I can also read it from another machine or my phone.

What I've learned after two months

The basic folder structure has held up. Most of my changes have been to the instructions and the amount of context agents load.

- **The index needs regular trimming.** My recent-log list grew to about fifty entries. Together, `AGENTS.md` and `HOME.md` reached roughly 4,700 tokens. Capping the list at ten entries and moving project-specific instructions onto their own pages brought that to about 3,000. Those are rough estimates using four characters per token.
- **Saving work needs an explicit step.** I packaged the write-back instructions as a skill so I can ask an agent to "log this to the hub." It also tells agents to save substantial work before finishing. I still check that the notes were saved.
- **Some files are better kept elsewhere.** I now have explicit conventions for finished documents: memos and specs can live in the hub, while decks, PDFs, and large reference collections get links to their source. When a document also lives in a shared drive, I record which copy is authoritative.

Here's how I run it now, followed by a template and a prompt you can use to set up your own.

How the files are organized

The hub is a wiki of short pages linked with `[[wiki-links]]`. I also browse it in Obsidian. Each page has a unique filename, so an agent can follow a link without needing the full path.

```
agent-hub/
├── AGENTS.md          # shared instructions
├── CLAUDE.md          # symlink to AGENTS.md
├── HOME.md            # index of projects and topics
├── projects/          # current state of each project
├── topics/            # notes used across projects
├── docs/              # memos, specs, and decision docs
├── work/              # working files, grouped by project
│   └── <project>/private/  # sensitive files, excluded from git
├── log/               # dated entries recording completed work
├── inbox/             # notes waiting to be filed
├── .agents/skills/    # skills for Codex
└── .claude/ .codex/ .cursor/   # tool-specific configuration
```

`AGENTS.md` tells agents how to read and update the hub. `HOME.md` tells them where to look. A project page holds its current state; dated log entries hold the history.

What agents read and save

At the start of a task, an agent reads `HOME.md` and follows the links relevant to the work. It shouldn't need to read the whole repo.

At the end of meaningful work, it:

1. Writes a log entry with what happened, the decisions and their reasons, and links to any outputs.
2. Updates the relevant project page.
3. Links the first mention of other projects or topics with `[[wiki-links]]`.
4. Updates `HOME.md` when it adds a page, keeping the recent-log list to about ten entries.
5. Asks before committing and pushing, unless I've already authorized it.

The `hub-log` skill carries these steps and is included in the template for Claude Code, Codex, and Cursor.

Finished memos and specs go in `docs/`. If a document is reviewed in a shared drive, its Markdown copy records the link in a `canonical:` field and says which copy to treat as authoritative. Decks, PDFs, and large reference collections stay where they are; the hub links to them.

Keeping project notes current

A page's “last updated” date doesn't tell me which facts were checked. I label individual items on project pages:

- `Current as of YYYY-MM-DD:` for an active fact or plan.
- `Decision (YYYY-MM-DD):` for a choice and its reason.
- `Open question as of YYYY-MM-DD:` for something unresolved.
- `Historical note (YYYY-MM-DD):` for background.
- `Needs verification:` for something to check before relying on it.

When a decision changes, I close the old entry with an end date and a link to its replacement:

```
Decision (2026-06-01 → 2026-08-10, superseded by [[new-page]]):
```

The instructions tell agents to verify current-state items older than 30 days before using them. Dates help identify what to check; they don't establish that a claim is still true.

For periodic cleanup, I ask an agent to “maintain the hub.” A librarian configuration tells it to fix broken links, file inbox notes, shorten overgrown pages, refresh the index, and flag stale projects. The template includes this configuration.

Keeping the context small

`AGENTS.md` holds the shared instructions, and `HOME.md` holds the index with about ten recent log entries. Project-specific instructions belong on the relevant project page, where an agent reads them only when needed.

A task then adds whichever project and topic pages it needs. A pre-commit hook rejects staged pages in those folders over 150 lines, prompting me or an agent to move older detail into the log. The hook limits page size; choosing which pages to read still depends on the index and the agent's instructions.

Sensitive files and access from other tools

Sensitive working files go in `work/<project>/private/`, which is excluded from git. They aren't uploaded through this repo and need a separate backup. This doesn't prevent a local agent with access from reading them, so I still choose what to share with each tool.

I read the synced notes through GitHub. A tool with authorized repo access can read them too. If a chat tool can't update the repo, it can draft a note for me to put in `inbox/` or pass to an agent that can save it.

I keep built-in tool memory enabled for preferences such as writing style. Project facts and decisions go in the hub so I can use them across tools.

If you put your hub in a private repo, don't assume a GitHub Pages site will be private too. Use the repo itself to read the notes.

Try it yourself

You can copy the template or ask an agent to create a hub from scratch.

Option 1: clone the repo

The template includes the folders, instructions, page-size hook, librarian, and write-back skill. Its `CHANGELOG.md` records updates for anyone who copied an earlier version. Copy just the `agent-hub-template/` folder into a new folder for your hub (the template README has download instructions). Inside that folder, run `git init`, then `git config core.hooksPath .githooks` to enable the page-size hook. Enable it again in each new clone. Commit the starter files, push to your own repo, and start writing.

The repo's instructions load when you work inside its folder. To make the hub available from your other projects, use this prompt to add a pointer in each tool's global settings:

```
Add a pointer to my agent hub to your own global configuration, not just this
project's:

- If you're Claude Code: add it to ~/.claude/CLAUDE.md (my user-level file, not this
  repo's), under a new "## Agent hub" heading.
- If you're Codex: add it to ~/.codex/AGENTS.md (my global instructions
  file), under a new heading.
- If you're Cursor: add it as a new global User Rule, not a project rule
  under .cursor/rules/ (that only fires in this one repo). If you cannot
  change User Rules directly, give me the exact text to paste into settings.

The pointer text:
"My agent hub lives at /Users/<you>/agent-hub. When a task touches an
ongoing project, a prior decision, or anything worth remembering across
sessions, read AGENTS.md there and follow it: start from HOME.md, then
write back at the end of meaningful work. Ask before committing and
pushing hub changes."

Show me the exact change before you save it -- this is a global file that
affects every future session, not just this one.
```

Replace `/Users/<you>/agent-hub` with your hub's absolute path. I initially put the pointer in Claude Code's project memory, which didn't help sessions in other projects. Use the global instructions file shown above.

Option 2: prompt an agent you already have

If you'd rather have your own Claude, Codex, or similar set this up for you from scratch, hand it this prompt as-is:

```
Set up a personal "agent hub": a git repo that serves as shared long-term memory
across all the AI agents I use, so project context carries between sessions and I
can access the saved notes from my phone too.

Create a new local folder and git repo with this structure:

  AGENTS.md         : root contract file (see contents below)
  CLAUDE.md         : symlink to AGENTS.md
  HOME.md           : index page, frontmatter: type: topic, status: active, updated: <today>
  projects/         : empty for now, one .md page per ongoing project later
  topics/           : empty for now, cross-cutting concept pages later
  docs/             : empty for now, finished memos/specs go here
  work/             : empty for now; work/<project>/ folders are created as work starts
  log/              : empty for now, append-only dated journal entries
  inbox/            : empty for now, quick-capture drops to be filed later
  .gitignore        : ignore: .DS_Store, work/*/private/, work/private/, private/

Write AGENTS.md with this contract (adapt wording to taste but keep the mechanics):

1. Every page is a small wiki page: frontmatter (type: project | topic | doc | log |
   inbox, status, updated: YYYY-MM-DD) followed by content. Cross-reference other pages with [[wiki-
links]]: a link
   resolves to a unique filename anywhere in the repo.
2. Reading protocol for any agent starting a task: read HOME.md first, then follow
   only the links relevant to the task. Never bulk-read the whole repo.
3. Writing protocol for any agent finishing a task: log the work to
   log/YYYY-MM-DD-short-slug.md (what happened, key decisions + why, what was
   produced and where); update the relevant projects/ page to reflect the new
   current state; link liberally, including to pages that don't exist yet;
   update HOME.md if a new page was created and keep its recent-log list to
   the newest ~10 entries. Ask before committing and pushing, unless I have
   already authorized it. Stage only files belonging to this task.
4. Freshness labels on facts inside project pages, not just a page-level date:
   "Current as of YYYY-MM-DD:", "Decision (YYYY-MM-DD):",
   "Open question as of YYYY-MM-DD:", "Historical note (YYYY-MM-DD):",
   "Needs verification:".
   When a fact is superseded or no longer true, close it with an end date and
   a pointer instead of deleting it, e.g.
   "Decision (2026-06-01 → 2026-08-10, superseded by [[new-page]]):".
   Verify unlabeled or uncertain facts, and current-state items older than
   30 days, before relying on them.
5. Sensitive content (PII, customer data, anything you wouldn't want on GitHub)
```

```
   goes in a private/ subfolder inside the relevant work/<project>/ folder —
   excluded from git and not uploaded through this repo. A local agent
   may still be able to read them.
6. Filenames: kebab-case, unique across the whole repo (this is what makes
   [[links]] unambiguous without a path).
7. Pages stay small: split anything that grows past ~150 lines into a
   distilled current-state page + a log entry holding the history. Add a
   pre-commit hook tracked at .githooks/pre-commit. Make it executable and
   enable it in each clone with `git config core.hooksPath .githooks`.
   The hook must fail the commit if any
   staged projects/*.md or topics/*.md page exceeds 150 lines. Check the
   staged version, not the working copy, including added, modified, and
   renamed pages; partial staging must not bypass the cap.
8. There is deliberately no "read the whole repo" script or skill: don't
   build one later for convenience. If HOME.md ever stops being a good
   enough index to answer "where do I look," that's the actual problem to
   fix, not a reason to add a bulk-read shortcut.
9. Other sessions, including mobile or cloud ones, may write to this repo
   too: git pull before making changes, not just before the final push, to
   avoid a rejected push or a conflict.

10. Finished memos, specs, and decision docs go in docs/<project>-<slug>.md
    with type: doc, status: draft | final, and updated: YYYY-MM-DD. Link them
    from the project page and log. If a shared-drive copy is canonical, put
    its link in canonical: frontmatter and state which copy is authoritative.
    Keep decks, PDFs, large prototypes, and documentation corpora at their
    source; the hub gets a pointer, not a duplicate.

Then:
- Set up per-tool config so every agent I use (Claude Code, Codex, Cursor,
  or others) reads AGENTS.md automatically at the start of a session, and
  follows the same read/write protocol above. Use each tool's native
  convention for this (Claude Code auto-reads CLAUDE.md at repo root; Codex
  and Cursor auto-read AGENTS.md there).
- Package the writing protocol as a skill named hub-log. Put the same SKILL.md in
  .claude/skills/hub-log/, .agents/skills/hub-log/, and .cursor/skills/hub-log/.
  Its description should fire on "log this to the hub" and proactively at
  the end of any substantial task; its body is step 3 above, spelled out.
- Add a global pointer too, not just the project-level file: edit
  ~/.claude/CLAUDE.md (Claude Code), ~/.codex/AGENTS.md (Codex), or add a global
  User Rule (Cursor; provide text to paste into settings if needed). Set
  this up for whichever tools I use, so sessions in other project folders
  know this hub exists. Use an absolute path, not ~, and
  show me the exact change before saving it, since these are global files
  that affect every future session, not just this one.
- Create a GitHub repo and push this as the origin, so it's reachable from
  mobile (Claude web/mobile, Codex cloud, or any other GitHub-aware agent).
```

```
- Optionally, define a small "librarian" helper (a subagent/config, whatever
  the tool supports) that on request: fixes broken [[links]], files inbox/
  items into the right pages, splits overgrown pages into log/ + distilled
  page, refreshes HOME.md (newest ~10 log entries), and flags projects
  untouched for 30+ days. It asks before committing and pushing unless
  already authorized.

Confirm the structure with me before making the first commit, and ask which
tools I actually use so you only wire up config for those.
```

Start with one real project page. Use it for a task, then check whether the agent saved enough context for the next session. You can add more topics and conventions as you need them.

If you try it, I'd like to hear what you change.